

# Generador de logs crudos masivos

Informe tecnico de benchmarking · Fase 1 del Proyecto Transversal  
Terminal Puerto Coquimbo, Sitio 3 · Programacion Avanzada

## Resumen

El simulador genera **10.000.014** eventos de telemetria (1,88 GB en JSON Lines, 187,8 bytes por evento) que cubren 9,699 dias de operacion continua del Sitio 3. Con ocho procesos trabajadores la corrida toma una mediana de **4,963 s**, equivalentes a 2.014.913 eventos/s y 378,3 MB/s efectivos. La validacion campo a campo de las 10.000.014 lineas no encontro ningun registro invalido.

El total no es exactamente diez millones porque el minimo del guion se interpreta como *piso*: con 179 puntos de medicion reportando cada quince segundos, el numero de ticks se redondea hacia arriba al tick completo, de modo que todos los sensores cubran exactamente la misma ventana temporal. De ahi los catorce eventos de mas. Los 187,8 bytes por evento estan por encima del rango de 130 a 170 que estima el guion porque ese rango corresponde al esquema minimo de seis campos, y el dataset emite ademas los cuatro opcionales recomendados: site, asset\_type, status y seq.

Estan implementadas las **dos** arquitecturas de escritura concurrente que admite el guion, detras de la misma interfaz, de modo que la eleccion se decide con una medicion y no con un argumento. Con la misma carga y los mismos trabajadores, los archivos segmentados terminan en 5,441 s y el escritor unico en 5,893 s: la **Opcion B es un 8,3 % mas rapida**, y los rangos de las dos series no se solapan.

Sobre el paralelismo, la conclusion es tajante: los hilos no mejoran en ningun punto —de 2 a 16 hilos el tiempo sube de 5,98 s a 6,21 s— mientras los procesos bajan de 3,12 s a 0,94 s sobre la misma carga. La curva de escalabilidad se mantiene casi ideal hasta cuatro procesos y se derrumba despues, y el informe muestra con mediciones que la causa no es el disco ni una fraccion secuencial fija, sino agotar los nucleos rapidos del procesador.

## 1. Entorno de pruebas

Todas las mediciones provienen de un unico equipo, sin otras aplicaciones en ejecucion. La arquitectura del procesador resulta ser el factor explicativo dominante de la seccion 9, de modo que conviene leerla con atencion.

| Componente           | Especificacion                                                                                                                  |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Procesador           | Intel Core i5-14600K, 3,5 GHz base                                                                                              |
| Nucleos fisicos      | 14 — <b>arquitectura hibrida</b> : 6 nucleos P (Performance, con hyperthreading) + 8 nucleos E (Efficiency, sin hyperthreading) |
| Nucleos logicos      | 20 (12 hilos sobre los P + 8 sobre los E)                                                                                       |
| Memoria RAM          | 15,78 GB                                                                                                                        |
| Almacenamiento       | Crucial P5 Plus, <b>SSD NVMe</b> de 1 TB                                                                                        |
| Sistema operativo    | Windows 11 Pro, build 26200                                                                                                     |
| Interprete           | CPython 3.14.4 (MSC v.1944, 64 bit), build clasico con GIL                                                                      |
| Arranque de procesos | spawn (unico disponible en Windows)                                                                                             |

**Por que importa que los nucleos sean heterogeneos.** Un nucleo E entrega del orden de la mitad del rendimiento monohilo de un nucleo P, y el planificador de Windows decide en cual corre cada proceso sin que el programa intervenga. La seccion 10 documenta una corrida en la que ese efecto se midio de forma directa y limpia.

## 2. Metodologia experimental

| Decision      | Como se aplica y por que                                                                                                                                                                                      |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Escala minima | Ningun experimento por debajo de 1.000.000 de eventos. La curva de escalabilidad usa 10.000.014 eventos fijos en las cinco configuraciones; a menor escala el ruido del sistema operativo domina la medicion. |

| Decision                        | Como se aplica y por que                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Calentamiento                   | Una corrida por configuracion se ejecuta y <b>se descarta</b> : la primera es sistematicamente mas lenta por el cache de paginas. Queda registrada en el CSV marcada como calentamiento, porque descartarla del analisis no es lo mismo que no registrarla.                                                                                                                                                                                                                                                                                                                           |
| Repeticiones                    | Tres corridas medidas por configuracion; se reporta la <b>mediana</b> , nunca la mejor. La configuracion de un solo trabajador se repitio seis veces mas como control, y ese control resultado revelador (seccion 10).                                                                                                                                                                                                                                                                                                                                                                |
| Aislamiento                     | Cada corrida se lanza como subprocesso independiente, con un interprete limpio. Durante toda la campana no se uso el equipo para nada mas.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Instrumentacion                 | El simulador emite su propia fila en bench/mediciones.csv al terminar cada corrida. Todas las figuras se construyen desde esos CSV; no hay ninguna captura de pantalla como evidencia.                                                                                                                                                                                                                                                                                                                                                                                                |
| Que se cronometra               | <b>Todas las tablas de este informe usan duracion_s</b> , que mide el trabajo de los procesos desde antes de crear el primero hasta cerrar el ultimo archivo. Los CSV de los barridos guardan ademas el tiempo de pared del subprocesso completo, que anade el arranque del interprete, argparse, la construccion del inventario y la escritura del manifiesto: en este equipo son unos 0,12 s, es decir un 2,3 % sobre una corrida de cinco segundos. Las figuras 3 y 4 se construyen desde esa segunda metrica y por eso quedan ligeramente por encima de la tabla de la seccion 5. |
| Deriva a lo largo de la campana | La campana completa dura cerca de una hora de carga sostenida y los tiempos derivan dentro de ella: la misma configuracion de ocho trabajadores marco 4,963 s a las 18:15 y 5,441 s a las 18:28. Por eso <b>los tiempos absolutos solo se comparan dentro de un mismo barrido</b> , nunca cruzando secciones. Cada barrido mide sus alternativas de forma consecutiva justamente para que la comparacion interna sea inmune a esa deriva.                                                                                                                                             |

### 3. Las dos arquitecturas, comparadas con datos

La **Opcion A** (--arch queue) separa calcular de escribir: los trabajadores encolan lotes de lineas ya serializadas y un unico proceso escritor mantiene el archivo abierto. La **Opcion B** (--arch shard) elimina el recurso compartido: cada trabajador escribe su propio part-NNN.jsonl. Ambas comparten el bucle de generacion y exponen la misma firma, de modo que la unica variable del experimento es como llegan los eventos al disco.

| Arquitectura       | Salida     | Mediana [s] | Eventos/s | Corridas medidas [s] |
|--------------------|------------|-------------|-----------|----------------------|
| B — N archivos     | 8 archivos | 5,441       | 1.837.900 | 5,358; 5,441; 5,501  |
| A — escritor unico | 1 archivo  | 5,893       | 1.696.931 | 5,869; 5,893; 6,025  |

**La Opcion B gana, y la diferencia no es ruido: su peor corrida (5,501 s) sigue siendo mejor que la mejor corrida de la Opcion A (5,869 s). Los rangos no se solapan.**

**Por que estas cifras no coinciden con las de la seccion 5.** Alli la configuracion de ocho trabajadores marca 4,963 s y aqui la misma arquitectura marca 5,441 s. La diferencia tiene dos componentes medibles: este barrido se ejecuto trece minutos despues, y la maquina habia acumulado casi media hora de carga sostenida (+9,6 %); ademas los CSV de barrido guardan tambien el tiempo de pared del subprocesso, un 2,3 % mayor, que es la cifra que aparece en versiones anteriores de este informe. Aqui se usa duracion\_s en las dos secciones. **La comparacion entre A y B no se ve afectada**: las dos series se midieron consecutivamente, con 42 segundos entre el ultimo shard y el primer queue, de modo que cualquier deriva las afecta por igual.

La ventaja es moderada pero consistente, y su origen es el que anticipa el guion: sobre un SSD NVMe, ocho descriptores en paralelo aprovechan mejor el ancho de banda que uno solo, porque el dispositivo atiende varias colas de peticiones a la vez. En un disco mecanico cabria esperar lo contrario, ya que varios flujos simultaneos obligarian al cabezal a saltar.

Conviene notar tambien lo que la medicion *no* muestra: el escritor unico no colapsa. Se mantiene a menos de un 10 % del otro patron porque los trabajadores serializan a texto *antes* de encolar, y el escritor solo ejecuta writelines sobre lineas ya listas. Si tuviera que serializar el, la diferencia seria de otro orden.

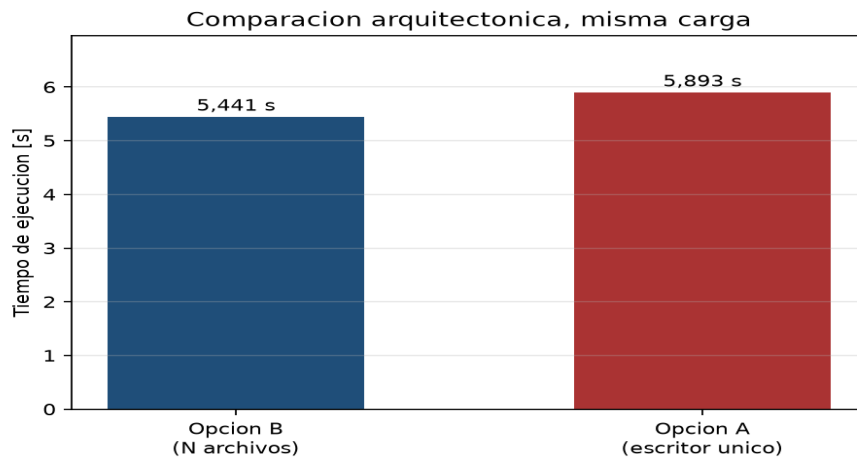


Figura 1. Tiempo de la corrida completa (10.000.014 eventos, 8 trabajadores, lote de 10.000) segun la arquitectura. Mediana de tres corridas medidas tras descartar una de calentamiento.

Como el rendimiento no es el unico criterio, la eleccion final pondera tambien las propiedades que difieren entre patrones:

| Criterio              | Opcion A (queue)                                                                                                        | Opcion B (shard)                                                                                                      |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Rendimiento medido    | 5,893 s                                                                                                                 | 5,441 s                                                                                                               |
| Reproducibilidad      | El mismo multiconjunto de eventos, distinto orden de lineas: el orden de llegada al escritor lo decide el planificador. | <b>Byte a byte identica</b> entre corridas con la misma semilla.                                                      |
| Costo de comunicacion | Un encurtido por lote encolado.                                                                                         | Dos por trabajador en toda la corrida: 16 con ocho procesos.                                                          |
| Modo de falla propio  | Desborde de RAM si falta contrapresion; bloqueo mutuo si muere el escritor. Exige supervision activa de los procesos.   | Shard truncado <i>en silencio</i> si muere un worker: el archivo sigue siendo JSON valido y solo el conteo lo delata. |
| Consumo en la Fase 2  | Un archivo unico; Spark lo divide, pero no reparte por archivo.                                                         | N archivos, que Spark reparte nativamente entre sus ejecutores.                                                       |

**Se entrega con** `--arch shard`: es la mas rapida en este equipo, la unica reproducible byte a byte y la que mejor consume Spark. La Opcion A queda implementada y verificada —`test_equivalencia_arquitecturas` comprueba que ambas producen el mismo multiconjunto de eventos—, de modo que la comparacion es reproducible y no una afirmacion.

#### 4. Hilos contra procesos: por que el GIL obliga a multiprocessing

El generador es aritmetica y manipulacion de objetos: actualizar el estado fisico del activo, formatear una marca de tiempo y serializar un diccionario. Es bytecode puro, sin llamadas bloqueantes, de modo que el *Global Interpreter Lock* nunca se libera. El experimento ejecuta la misma carga de 2.000.146 eventos contra SumideroNulo, que cuenta las lineas sin escribirlas: al eliminar el disco, cualquier diferencia es atribuible al modelo de concurrencia.

| Modo       | n  | Mediana [s] | Eventos/s totales | Frente a su propio n=1 |
|------------|----|-------------|-------------------|------------------------|
| secuencial | 1  | 12,186      | 164.135           | 1,00×                  |
| hilos      | 1  | 12,189      | 164.094           | 1,00×                  |
| hilos      | 2  | 5,977       | 334.640           | 2,04×                  |
| hilos      | 4  | 6,019       | 332.305           | 2,03×                  |
| hilos      | 8  | 6,115       | 327.088           | 1,99×                  |
| hilos      | 16 | 6,209       | 322.137           | 1,96×                  |
| procesos   | 1  | 6,242       | 320.434           | 1,00×                  |

| Modo     | n  | Mediana [s] | Eventos/s totales | Frente a su propio n=1 |
|----------|----|-------------|-------------------|------------------------|
| procesos | 2  | 3,124       | 640.252           | 2,00×                  |
| procesos | 4  | 1,634       | 1.224.080         | 3,82×                  |
| procesos | 8  | 1,085       | 1.843.453         | 5,75×                  |
| procesos | 16 | 0,936       | 2.136.908         | 6,67×                  |

Los hilos estan **planos y empeorando**: de 2 a 16 hilos el tiempo sube monotonamente de 5,977 s a 6,209 s. Anadir hilos no solo no ayuda, cuesta: son cambios de contexto y contencion por el candado sobre un trabajo que nunca lo suelta. Los procesos, con la carga identica, bajan de 6,242 s a 0,936 s. Con 16 unidades de concurrencia los procesos terminan **6,6 veces mas rapido** que los hilos en bruto; el recuadro siguiente descuenta el sesgo de medicion y deja la ventaja real en 3,4×

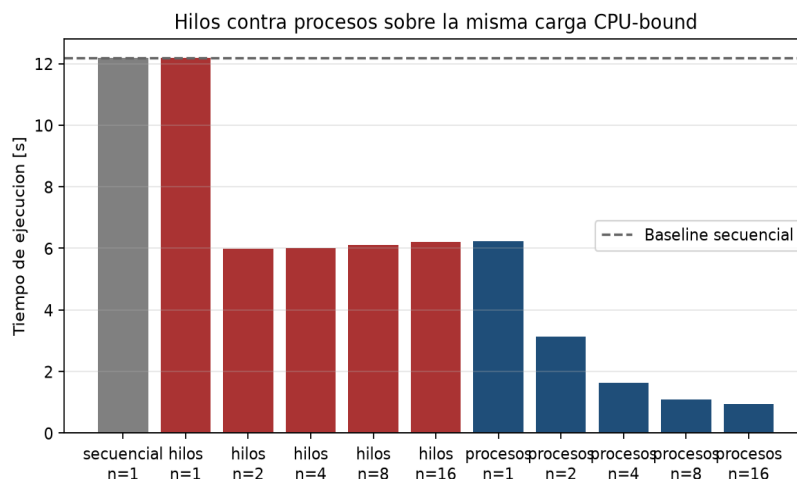


Figura 2. Misma carga CPU-bound, sumidero nulo. Las barras rojas (hilos) no bajan; las azules (procesos) si. La linea punteada es el baseline secuencial.

**Un sesgo de medicion que conviene declarar, y que se puede cuantificar.** El modo secuencial marca 12,186 s y procesos n=1 marca 6,242 s ejecutando exactamente el mismo trabajo con una sola unidad de concurrencia. La diferencia no esta en el codigo sino en donde corre: los modos *secuencial* e *hilos* se cronometran dentro del proceso padre, y los *procesos* en hijos recién creados, que el planificador de Windows coloca en nucleos distintos.

Ese cociente,  $12,186 / 6,242 = 1,95\times$ , es la magnitud del sesgo, y permite corregir la comparacion en lugar de renunciar a ella. Con 16 unidades de concurrencia el cociente bruto entre hilos y procesos es 6,63×

descontado el sesgo, la ventaja real de los procesos es de **3,4×**. El argumento principal, en todo caso, no depende de ningun cociente: los hilos no mejoran al anadir mas y los procesos si, y eso es robusto frente a cualquier desplazamiento comun.

## 5. Curva de escalabilidad: speedup y eficiencia

Total de eventos fijo en 10.000.014 en las cinco configuraciones. La linea base  $T(1) = 31,000$  s es la mediana de las tres corridas del protocolo, con una desviacion tipica de 0,303 s. La seccion 10.1 explica por que esa eleccion es ademas la conservadora.

| n  | Mediana T(n) [s] | Desv. [s] | S(n) | E(n) | Eventos/s | MB/s  | Corridas medidas [s] |
|----|------------------|-----------|------|------|-----------|-------|----------------------|
| 1  | 31,000           | 0,303     | 1,00 | 1,00 | 322.581   | 60,6  | 30,84; 31,00; 31,43  |
| 2  | 15,839           | 0,233     | 1,96 | 0,98 | 631.354   | 118,5 | 15,72; 15,84; 16,17  |
| 4  | 8,129            | 0,053     | 3,81 | 0,95 | 1.230.165 | 231,0 | 8,07; 8,13; 8,18     |
| 8  | 4,963            | 0,032     | 6,25 | 0,78 | 2.014.913 | 378,3 | 4,91; 4,96; 4,97     |
| 16 | 3,958            | 0,027     | 7,83 | 0,49 | 2.526.532 | 474,4 | 3,94; 3,96; 4,00     |

La eficiencia se mantiene practicamente ideal hasta dos procesos (0,98), sigue alta con cuatro (0,95) y se derrumba despues: 0,78 con ocho y 0,49 con dieciseis. Ningun punto supera 1, que es lo que debe ocurrir cuando la linea base esta bien medida. La dispersion, ademas, decrece de forma sistematica con n: de 0,303 s con un trabajador a 0,027 s con dieciseis, porque al repartir la carga entre mas procesos la mezcla de tipos de nucleo se promedia dentro de la propia corrida.

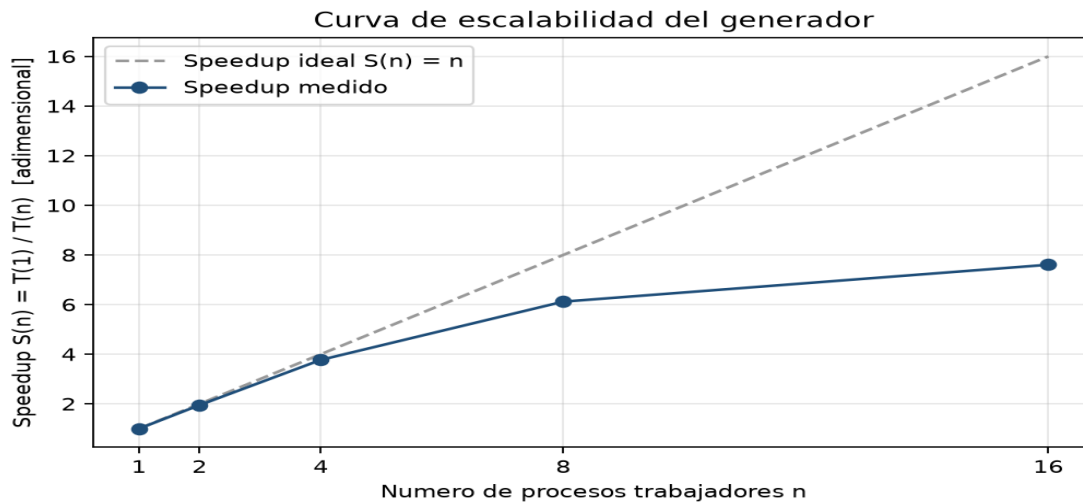


Figura 3. Speedup medido frente a la recta ideal  $S(n)=n$ . La distancia entre ambas es el objeto de la seccion 9.

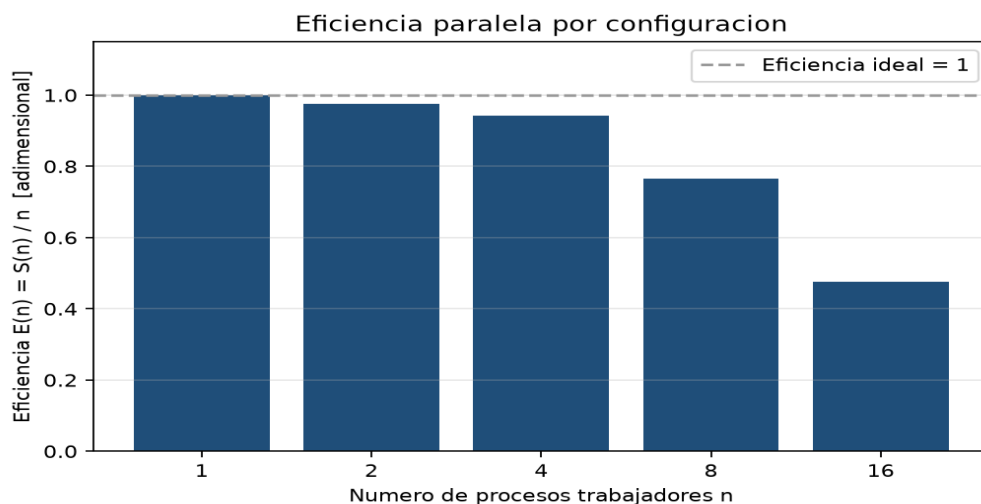


Figura 4. Eficiencia paralela  $E(n)=S(n)/n$ .

## 6. Rendimiento de entrada/salida

El rendimiento de escritura crece con el paralelismo hasta 474,4 MB/s con dieciseis trabajadores. Esa cifra es muy inferior a lo que sostiene un SSD NVMe moderno, lo que ya sugiere que el almacenamiento no es el factor limitante; la seccion 9 lo confirma con un experimento que quita el disco de la ecuacion por completo.

| n  | Eventos/s | MB/s efectivos | Eventos/s por trabajador |
|----|-----------|----------------|--------------------------|
| 1  | 322.581   | 60,6           | 322.581                  |
| 2  | 631.354   | 118,5          | 315.677                  |
| 4  | 1.230.165 | 231,0          | 307.541                  |
| 8  | 2.014.913 | 378,3          | 251.864                  |
| 16 | 2.526.532 | 474,4          | 157.908                  |

La ultima columna anticipa el analisis: el rendimiento *por trabajador* cae de forma sostenida a medida que se agregan procesos, y esa caída es la que aplana la curva agregada.

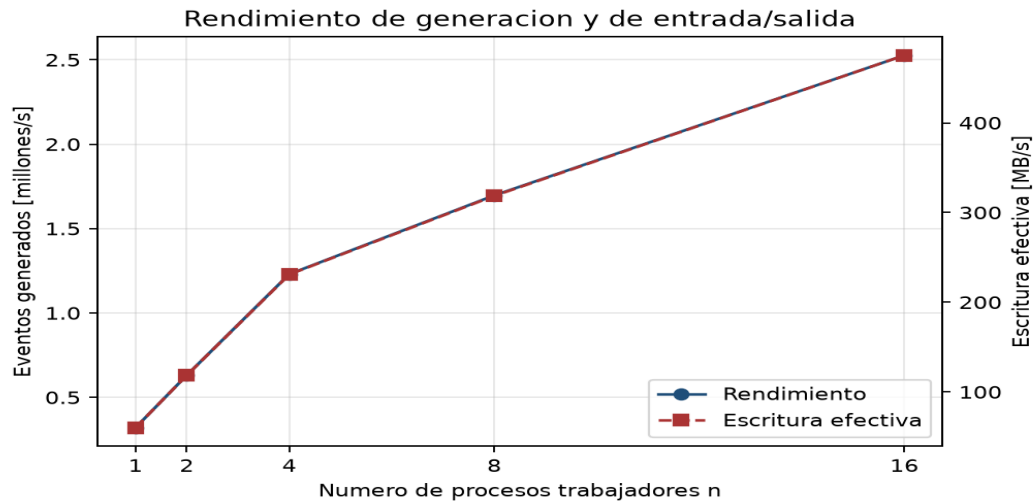


Figura 5. Rendimiento de generacion (millones de eventos/s) y escritura efectiva (MB/s) por configuracion.

## 7. Costo de serializacion entre procesos

Dos procesos no comparten memoria: pasarse un objeto exige serializarlo, escribirlo en un pipe y deserializarlo al otro lado. Se midio con dos experimentos complementarios que conviene no confundir.

### 7.1 El canal aislado

bench/micro\_pickle.py monta una cola, un productor y un consumidor, sin fisica, sin disco y sin formateo, con un millon de eventos por configuracion. El tiempo medido es atribuible al canal IPC y a nada mas.

| Eventos por put() | Operaciones put() | Mediana [s] | Eventos/s | Frente al peor caso |
|-------------------|-------------------|-------------|-----------|---------------------|
| 1                 | 1.000.000         | 13,134      | 76.138    | 1×                  |
| 100               | 10.000            | 0,214       | 4.678.778 | 61×                 |
| 1.000             | 1.000             | 0,124       | 8.065.128 | 106×                |
| 10.000            | 100               | 0,117       | 8.573.116 | 113×                |

### 7.2 El simulador completo

El barrido de lotes mide lo mismo *dentro* del simulador con `--arch queue`, donde el costo del canal compite con la generacion y la escritura. 1.000.073 eventos y ocho trabajadores.

| --batch-size | Operaciones put() | Mediana [s] | Corridas medidas [s]   | Frente al mejor |
|--------------|-------------------|-------------|------------------------|-----------------|
| 1            | 1.000.000         | 11,425      | 11,281; 11,425; 11,458 | 15,7× mas lento |
| 100          | 10.000            | 0,800       | 0,792; 0,800; 0,812    | 1,1× mas lento  |
| 1.000        | 1.000             | 0,729       | 0,728; 0,729; 0,757    | 1,0× mas lento  |
| 10.000       | 100               | 0,809       | 0,803; 0,809; 0,810    | 1,1× mas lento  |

Encolar evento por evento (fila resaltada) hace la corrida **15,7 veces mas lenta** que el mejor lote. El optimo esta en 1.000 eventos por mensaje, y a partir de ahi crecer deja de compensar: con 10.000 el tiempo vuelve a subir levemente. **Mas grande no es automaticamente mejor**, porque el lote retiene mas memoria y aumenta la latencia con que el escritor recibe trabajo.

La diferencia entre 1.000 y 10.000 es de solo un 11 %, asi que conviene comprobar que no es ruido antes de afirmar un optimo. Las corridas individuales lo confirman: 1.000 ocupa el rango 0,728–0,757 s y 10.000 el rango 0,803–0,810 s. **No se solapan**, de modo que la ordenacion esta respaldada al mismo nivel que el resto del informe.

**Por que la entrega usa `--batch-size 10000` si el optimo de esta tabla es 1.000.** Este barrido corre sobre `--arch queue`, donde el lote gobierna el tamaño del put y por tanto el compromiso entre costo de encurtido y latencia hacia el escritor. La configuracion de entrega es `--arch shard`, donde no hay cola: ahi el lote solo

decide cada cuantas lineas se llama a writelines sobre un bufer de 4 MB, y el compromiso es otro. Los dos optimos no tienen por que coincidir.

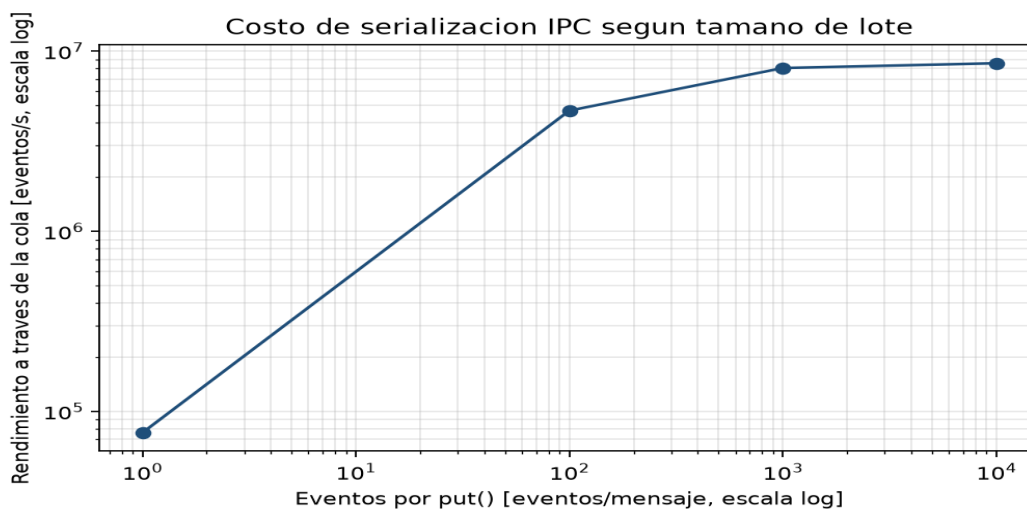


Figura 6. Canal IPC aislado, ambos ejes en escala logaritmica.

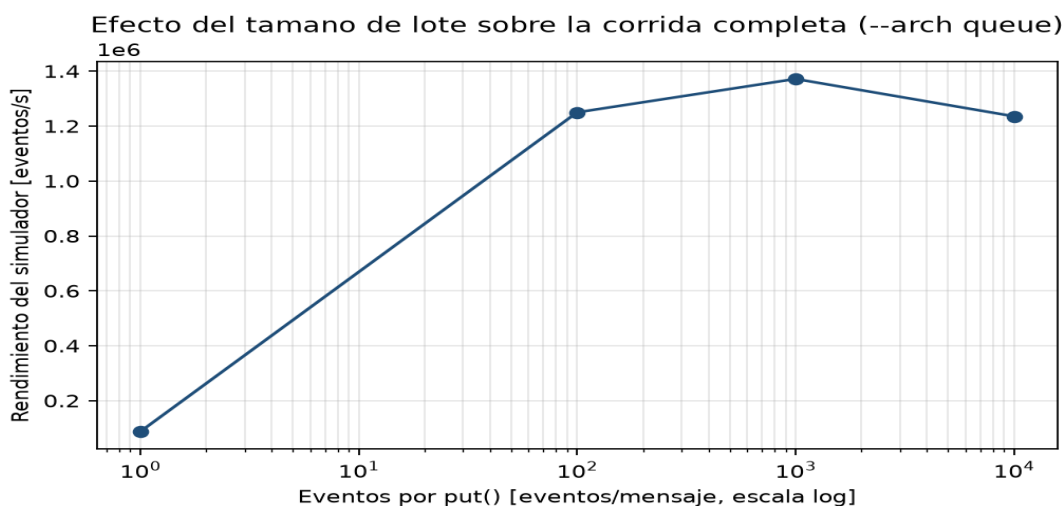


Figura 7. El mismo efecto sobre el simulador completo con --arch queue.

## 8. Gestion de memoria

### 8.1 Consumo maximo observado

Se midio muestreando el conjunto de trabajo de todos los procesos python cada 80 ms durante corridas completas de diez millones de eventos, y reteniendo el maximo.

| Configuracion       | Procesos | Pico de un proceso | Pico del conjunto | Sobre 15,78 GB |
|---------------------|----------|--------------------|-------------------|----------------|
| shard (entrega)     | 9        | 29,7 MB            | 258,1 MB          | 1,6 %          |
| queue, maxsize=8    | 10       | 32,4 MB            | 275,7 MB          | 1,7 %          |
| queue, maxsize=1024 | 10       | 32,5 MB            | 305,9 MB          | 1,9 %          |

El recuento de procesos confirma la estructura de cada arquitectura: nueve con shard (el padre y ocho trabajadores) y diez con queue, que suma el proceso escritor. Ese proceso extra explica los ~18 MB de diferencia entre las dos primeras filas.

**La memoria nunca fue una restriccion: la configuracion de entrega usa 258 MB, un 1,6 % de la RAM del equipo.**

### 8.2 Que maxsize se eligio, y por que

La cota de la cola fija a la vez el techo de RAM y el margen que tienen los trabajadores para adelantarse al escritor. Se midieron las dos caras del compromiso.

| --queue-maxsize | Mediana [s] | Eventos/s | Techo teorico de la cola | Pico real observado |
|-----------------|-------------|-----------|--------------------------|---------------------|
| 1               | 16,303      | 613.385   | 1,9 MB                   | —                   |
| 8               | 7,556       | 1.323.453 | 15,0 MB                  | 275,7 MB            |
| 64              | 7,520       | 1.329.789 | 120,2 MB                 | —                   |
| 512             | 7,581       | 1.319.089 | 961,4 MB                 | —                   |

Con maxsize=1 la corrida tarda 16,303 s frente a 7,520 s, un **2,17× mas lenta**: tanta contrapresion asfixia al escritor, que se queda sin trabajo pendiente mientras los productores esperan hueco. Desde 8 en adelante el tiempo es plano, de modo que subir la cota solo compra techo de memoria sin ninguna ganancia. **64 esta comodamente en la meseta** y es el valor elegido.

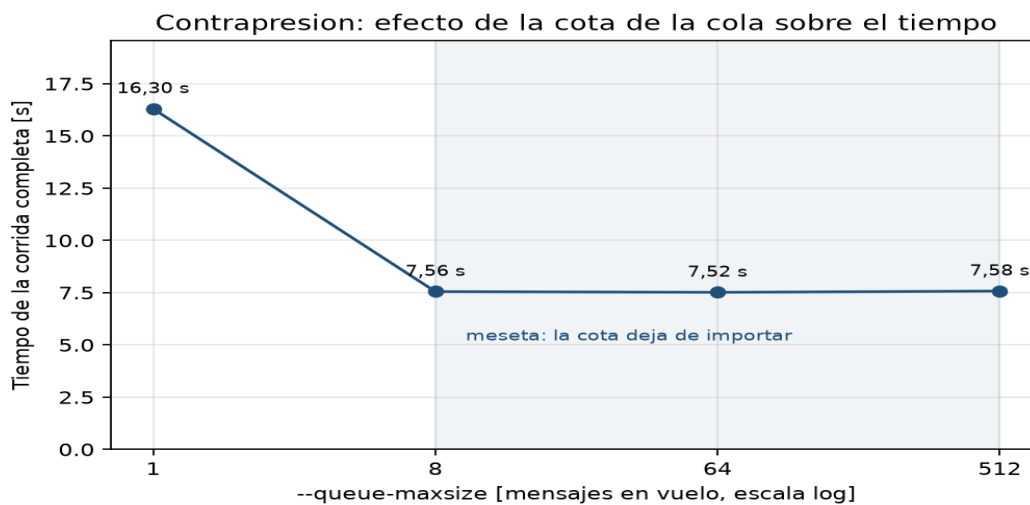


Figura 8. Efecto de la cota de la cola sobre el tiempo total. Por debajo de 8 la contrapresion penaliza; por encima, la cota deja de importar.

### 8.3 Sobre el desbordamiento

El guion pide contarle si se llega a provocar. **No se provoco**, y el motivo es informativo. La corrida con maxsize=1024 de la sección 8.1 tiene un techo teorico de 1,92 GB y su pico real fue de 306 MB: frente a maxsize=8, multiplicar la cota por 128 solo costo 30 MB. La razón es que maxsize es un *techo*, no una reserva. La cola solo se llena si el consumidor se queda atras, y en este equipo el escritor nunca se queda atras porque el NVMe absorbe todo lo que los ocho trabajadores producen.

El mecanismo del desborde, por tanto, esta entendido y acotado aunque no se haya alcanzado: para provocarlo haria falta un consumidor mas lento que los productores —un disco mecanico, un destino de red, o un escritor que ademas serializara—, y en ese caso la RAM crece a la diferencia entre ambos ritmos hasta el techo. La proteccion es que, al llegar a la cota, put bloquea y los trabajadores se frenan solos. **La contrapresion no es una limitacion del patron: es lo que lo hace utilizable.**

## 9. Interpretacion critica: donde se aplanla la curva y por que

La curva pierde eficiencia entre cuatro y ocho procesos, y se degrada de nuevo hacia dieciseis. Hay cuatro sospechosos habituales y las mediciones permiten descartar tres.

### 9.1 No es el disco

La prueba mas directa es que el experimento de la sección 4 escribe a SumideroNuLo: no toca el disco en absoluto. Y aun asi presenta exactamente la misma degradacion. Calculando el rendimiento por trabajador de la serie de procesos, sin disco:



| n  | Eventos/s por worker (sin disco) | Frente a n=1 | Eficiencia | f de Amdahl implicita |
|----|----------------------------------|--------------|------------|-----------------------|
| 1  | 320.434                          | 1,00×        | 1,00       | —                     |
| 2  | 320.126                          | 1,00×        | 1,00       | 0,1 %                 |
| 4  | 306.020                          | 0,96×        | 0,96       | 1,6 %                 |
| 8  | 230.432                          | 0,72×        | 0,72       | 5,6 %                 |
| 16 | 133.557                          | 0,42×        | 0,42       | 9,3 %                 |

El rendimiento individual es **plano hasta cuatro procesos** —cada trabajador sostiene alrededor de 320.000 eventos/s— y cae despues. Si el disco fuera la causa, esta serie, que no escribe nada, deberia escalar limpiamente. No lo hace.

Hay una segunda prueba, mas directa todavia, que sale de comparar esta tabla con la de la seccion 6. En las configuraciones de uno, dos y cuatro trabajadores, el rendimiento por trabajador escribiendo 1,88 GB al disco y sin escribir nada difiere en menos de un 1,5 %:

| n | Con disco (§6) | Sin disco (§9.1) | Diferencia |
|---|----------------|------------------|------------|
| 1 | 322.581        | 320.434          | +0,7 %     |
| 2 | 315.677        | 320.126          | −1,4 %     |
| 4 | 307.541        | 306.020          | +0,5 %     |

Escribir el dataset completo cuesta, en esas configuraciones, menos que el ruido de la medicion. La comparacion deja de ser limpia en n=8 y n=16, donde las dos series divergen hasta un 18 %: las cargas son distintas —10.000.014 eventos contra 2.000.146— y el costo fijo de arranque se amortiza de forma muy diferente cuando el tiempo total baja de un segundo. Por eso solo se citan las tres primeras filas. A ello se suma la seccion 3: si el almacenamiento saturara, escribir un archivo y escribir ocho no darian tiempos a menos de un 10 % de distancia.

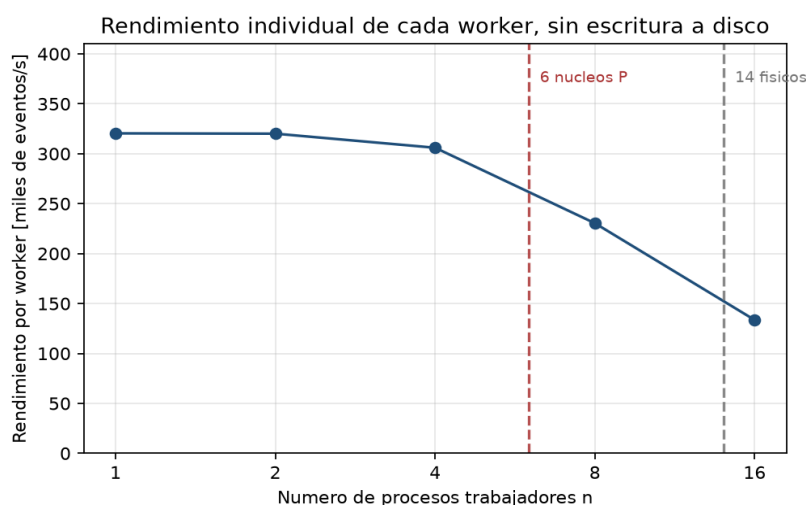


Figura 9. Rendimiento individual de cada trabajador con el disco fuera de la ecuacion. La meseta termina justo donde se agotan los nucleos P.

## 9.2 No es la ley de Amdahl

La ultima columna de la tabla anterior es la prueba. Si el aplanamiento se debiera a una fraccion secuencial fija, esa fraccion seria **la misma** calculada desde cualquier punto de la curva, porque es una constante del programa. No lo es: crece de forma sostenida con el numero de procesos.

**Una fraccion secuencial que crece con n no es una fraccion secuencial: es un recurso que se acaba. El programa no tiene mas codigo serie con 16 procesos que con 4; lo que tiene son menos nucleos rapidos disponibles.**

## 9.3 Es la maquina: nucleos rapidos que se agotan

| Candidato                           | Veredicto                | Evidencia                                                                                                                                                                                                                                                                                                       |
|-------------------------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Saturacion del disco                | <b>Descartado</b>        | La serie sin disco de la seccion 9.1 presenta la misma degradacion. Ademas, una arquitectura de un archivo y otra de ocho quedan a menos de un 10 % entre si.                                                                                                                                                   |
| Fraccion secuencial (Amdahl)        | <b>Descartado</b>        | La f implicita no es constante: crece del orden del 2 % en n=2 al 7 % en n=16. Un valor que depende del punto de ajuste describe un limite de recursos, no una seccion serie.                                                                                                                                   |
| Costo de arranque con spawn         | <b>Contribuye, menor</b> | Esta dentro del cronometro. Con 16 procesos son 16 interpretes que reimportan el modulo, un costo fijo repartido sobre un tiempo total cada vez menor, de modo que su peso relativo crece.                                                                                                                      |
| Nucleos fisicos y su heterogeneidad | <b>Causa dominante</b>   | El equipo tiene 6 nucleos P y 8 nucleos E. El rendimiento por trabajador es plano mientras caben en los P y cae en cuanto hay que usar los E, que rinden del orden de la mitad. Con 16 procesos se superan ademas los 14 nucleos fisicos y se compite por unidades de ejecucion compartidas via hyperthreading. |

La seccion 10 aporta la confirmacion mas nitida de esa causa: una misma configuracion, repetida nueve veces, produjo dos grupos de tiempos separados por un factor de casi dos, que es exactamente la relacion de rendimiento entre los dos tipos de nucleo.

**Conclusion operativa.** El punto de operacion util de este equipo esta en ocho trabajadores. Pasar a dieciseis todavia reduce el tiempo un 20 %, pero duplicando los procesos y con la eficiencia por debajo de 0,49. A partir de ahi la maquina no tiene mas nucleos rapidos que ofrecer.

## 10. Amenazas a la validez

### 10.1 La linea base T(1) es bimodal

T(1) es el denominador de toda la serie de speedup, asi que esa configuracion se repitio seis veces mas, ademas de las tres del protocolo. El resultado no fue ruido alrededor de una media: fueron **dos grupos separados**.

| Grupo  | Corridas | Tiempos [s]         | Mediana [s] |
|--------|----------|---------------------|-------------|
| Rapido | 3        | 30,83; 30,99; 31,18 | 30,99       |
| Lento  | 3        | 58,94; 59,55; 60,24 | 59,55       |

Tres corridas en torno a 31,0 s y tres en torno a 59,5 s, con un factor de **1,92×** entre modos y practicamente ninguna dispersion dentro de cada grupo. Un proceso unico, que Windows trata como carga de fondo, acaba en un nucleo P o en uno E segun lo decida el planificador, y ahi se queda toda la corrida. El factor medido coincide con la relacion de rendimiento entre ambos tipos de nucleo, y es la evidencia mas directa del informe para la causa identificada en la seccion 9.

Las tres corridas del protocolo cayeron todas en el grupo rapido, de modo que el reparto final de las nueve es de **seis rapidas y tres lentas**. Se adopta la mediana del grupo rapido,  $T(1) = 31,000$  s con desviacion 0,303 s, y conviene subrayar que es la eleccion **conservadora**: un denominador menor produce los speedup mas bajos de los dos posibles. Si se hubiera tomado el grupo lento, todos los speedup del informe serian casi el doble y aparentemente superlineales, lo que habria sido un artefacto y no un merito.

Con dos o mas procesos el efecto se atenua en lugar de partir la distribucion, porque la mezcla de tipos de nucleo se promedia dentro de la misma corrida: de ahi que la dispersion caiga monotonamente al aumentar n. **Por eso el analisis de la seccion 9 se apoya en el rendimiento por trabajador y no en S(n)**, que arrastra esta sensibilidad.

### 10.2 Lo que no se pudo medir en este equipo

| Medicion                       | Por que no                                                                                                                                        | Como se cubrio                                                                                                                                                             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pico de RSS desde el simulador | src/main.py instrumenta la memoria con resource, que es POSIX y no existe en Windows; las columnas pico_*_mb del CSV quedan vacias.               | Se midio externamente muestreando el conjunto de trabajo de todos los procesos cada 80 ms (seccion 8.1), que ademas cubre a los hijos y no solo al padre.                  |
| Encurtidos de --arch queue     | Con spawn el hijo levanta un interprete limpio y el parche que cuenta las llamadas, aplicado en el padre, no viaja. Los put ocurren en los hijos. | El numero de encurtidos se deriva del tamano de lote y su costo se acota con el micro-benchmark de la seccion 7.1.                                                         |
| Afinidad de procesador         | No se instrumentaron contadores de rendimiento ni asignacion de nucleos.                                                                          | La hipotesis de los nucleos E se sostiene sobre la bimodalidad medida en 10.1 y sobre la coincidencia entre los codos observados y el recuento de nucleos P y fisicos.     |
| Escritura fisica confirmada    | El cierre de archivo no fuerza vaciado a disco, asi que parte de los 1,88 GB puede quedar en el cache de escritura al parar el cronometro.        | No altera las conclusiones —si acaso refuerza que el disco no limita—, pero las cifras de MB/s son de rendimiento observado por la aplicacion, no de escritura confirmada. |

## 11. Conclusiones

**1. El paralelismo por procesos era obligatorio y quedo demostrado.** Sobre carga CPU-bound los hilos no mejoran en ningun punto —de 2 a 16 hilos el tiempo sube de 5,98 a 6,21 s—. Descontando el sesgo de medicion cuantificado en la seccion 4, los procesos terminan 3,4 veces por delante de los hilos con la misma concurrencia.

**2. La meta de volumen se cumple con holgura.** 10.000.014 eventos y 1,88 GB en 4,96 s con ocho trabajadores (2.014.913 eventos/s), con cero registros invalidos en la validacion campo a campo del dataset completo.

**3. La Opcion B es la mas rapida, y se eligio con datos.** 5,441 s contra 5,893 s, con rangos que no se solapan. A la ventaja de rendimiento se suman la reproducibilidad byte a byte y el reparto nativo entre ejecutores de Spark en la Fase 2.

**4. El cuello de botella no es el disco.** La serie que escribe a un sumidero nulo presenta la misma degradacion que la que escribe 1,88 GB, y hasta cuatro trabajadores la diferencia entre escribir y no escribir es inferior al 1,5 %. Ademas, dos arquitecturas de escritura radicalmente distintas quedan a menos de un 10 % entre si.

**5. Tampoco es Amdahl.** La fraccion secuencial implicita no es constante: crece de 0,1 % a 9,3 % segun el punto desde el que se ajuste, casi dos ordenes de magnitud. Una fraccion que depende de n describe un recurso que se agota, y ese recurso son los 6 nucleos P del procesador.

**6. El costo de comunicacion es de dos ordenes de magnitud.** El canal aislado rinde 113 veces mas con lotes de 10.000 que evento por evento, y sobre el simulador completo la diferencia sigue siendo de 15,7×. El optimo no es el lote mas grande sino uno intermedio.

**7. La memoria nunca fue una restriccion, y el parametro que la gobierna quedo justificado.** 258 MB en la configuracion de entrega, un 1,6 % de la RAM. El barrido muestra que por debajo de 8 la contrapresion penaliza el tiempo y por encima solo compra techo inutil.

**8. La medicion en procesadores hibridos exige cautela.** La misma configuracion, repetida nueve veces, dio dos grupos de tiempos separados por un factor de casi dos. No es ruido: es en que tipo de nucleo cae el proceso. Se documenta en lugar de promediarlo, y se elige la linea base conservadora.

**Reproducibilidad.** Los datos crudos de todas las corridas, incluidas las de calentamiento, estan en bench/mediciones.csv, con una fila por ejecucion. Los barridos dejan ademas su propio resumen (escalabilidad.csv, comparacion\_arquitecturas.csv, comparacion\_lotes.csv, hilos\_vs\_procesos.csv, mediciones\_pickle.csv, memoria\_observada.csv). Dos ejecuciones de --arch shard con la misma semilla producen datasets identicos byte a byte.